

# Microservices Architecture

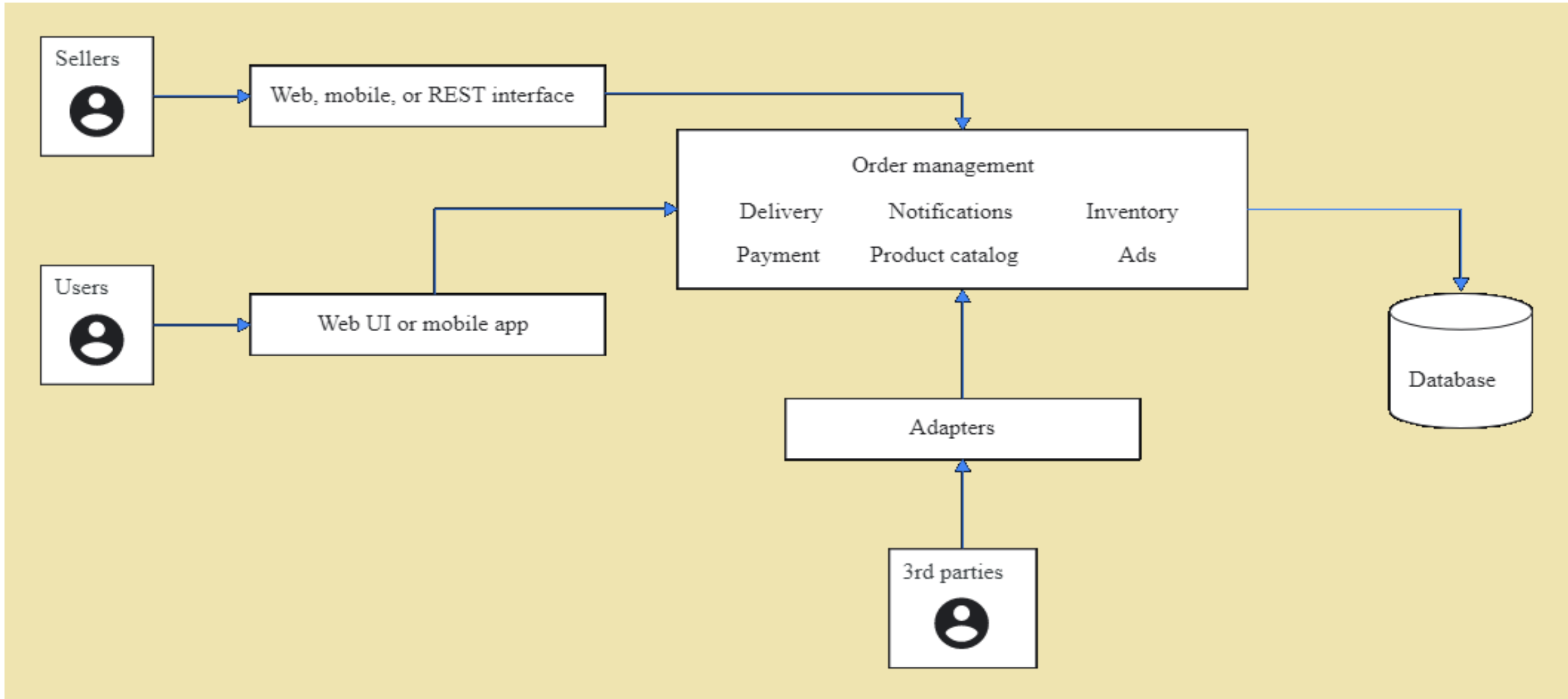
# Monolithic Architecture

- Widely used **architectural pattern**, traditional way of building applications
- It is designed in a way that the entire application built is ultimately a **single piece**, no matter how much it is decoupled by using Patterns and tiers – 1/2/3.
- All the services or business functionalities would be **tightly coupled** within the single Solution.
- The major thing to note is that while publishing the application, it is **deployed to a single server** only.

# Monolithic Architecture

- An **e-commerce application** built as a monolith might include:
  - User management
  - Product catalog
  - Order processing
  - Payment handling etc.
- All of these are packaged and deployed **together** as one application.

# Monolithic E-Commerce Application Architecture



# Monolithic Benefits

- End-to-end **testing** is easy
- Straightforward **debugging** (everything in one place)
- Single packaged application can be **deployed easily** on the server
- **Single solution** to cross-cutting concerns (logging, caching, security are easy because application share all the resources)
- **Performance** advantage

# Monolithic Drawbacks (..cont)

- Progressively become **harder to build and debug**
- **Changes are difficult** in complex and tightly coupled modules
- **Testing** process is **time consuming**
- **Complicated** to do continuous integration and deployment **CI/CD**
- Single update will required **redeploying** an entire application
- Lot of **manual testing** required to do regression

# Monolithic Drawbacks (..cont)

- Difficult to **scale**
- A bug in one of the module may **bring down the entire system**
- Difficult to adopt/upgrade to a **new technologies/framework**

# Microservices Architecture

- It is an **architectural style** that structures an application as a **collection of services** that are
  - Highly **maintainable and testable**
  - **Loosely coupled**
  - **Independently deployable**
  - Organized around **business capabilities**
  - Owned by a **small team**
- It enables the **rapid, frequent and reliable delivery** of large, complex applications
- It also enables an organization to **evolve its technology stack**



# Companies Using Microservices

amazon.com

NETFLIX

GILT



ebay



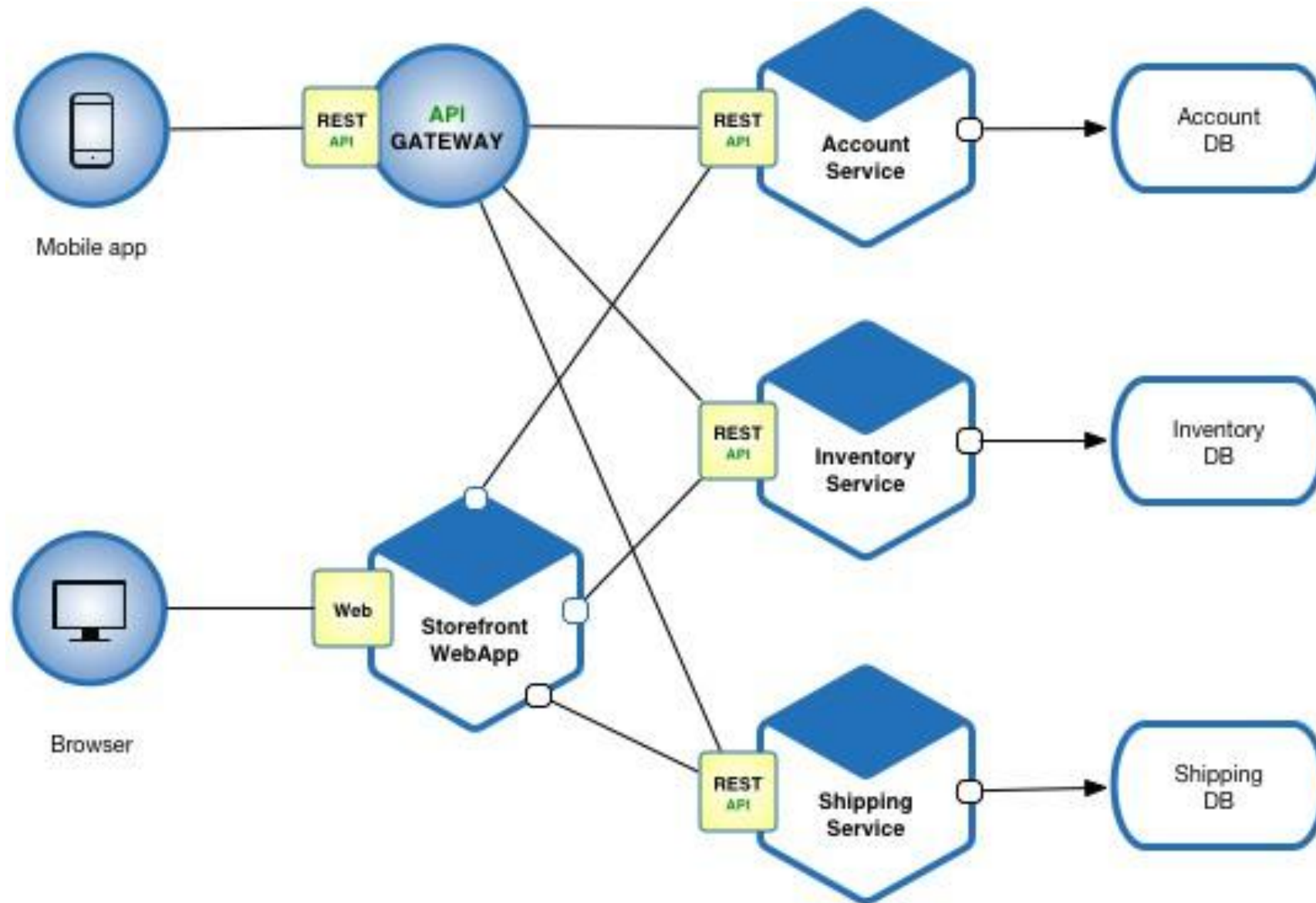
NORDSTROM

theguardian

# Characteristics of Microservices Architecture

- **Small** in size
- **Messaging** enabled
- Bounded by **contexts**
- **Autonomously** developed
- **Independently** deployable
- **Decentralized**
- **Built and released** with automated processes

# E-Commerce Application



# Microservices Architecture Design

- Distributed architecture

- All the services communicate with the **API gateway** through REST or RPC.
- These services can be deployed as **multiple instances**, and the **requests can be distributed** to these instances

- Separately deployed components

- Each component is deployed separately.
- If one component needs changes, others don't have to be deployed again

# Microservices Architecture Design (..cont)

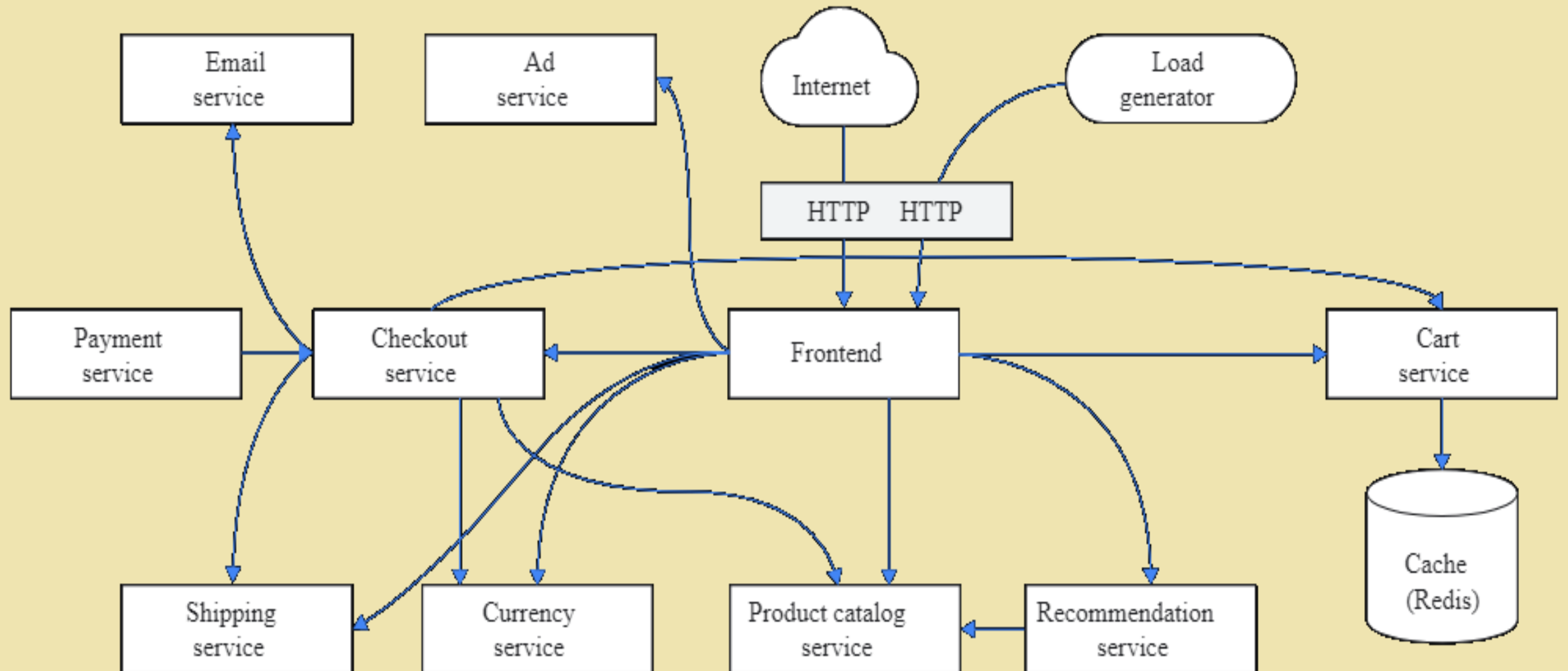
- Service components

- Services components communicate with each other via service discovery

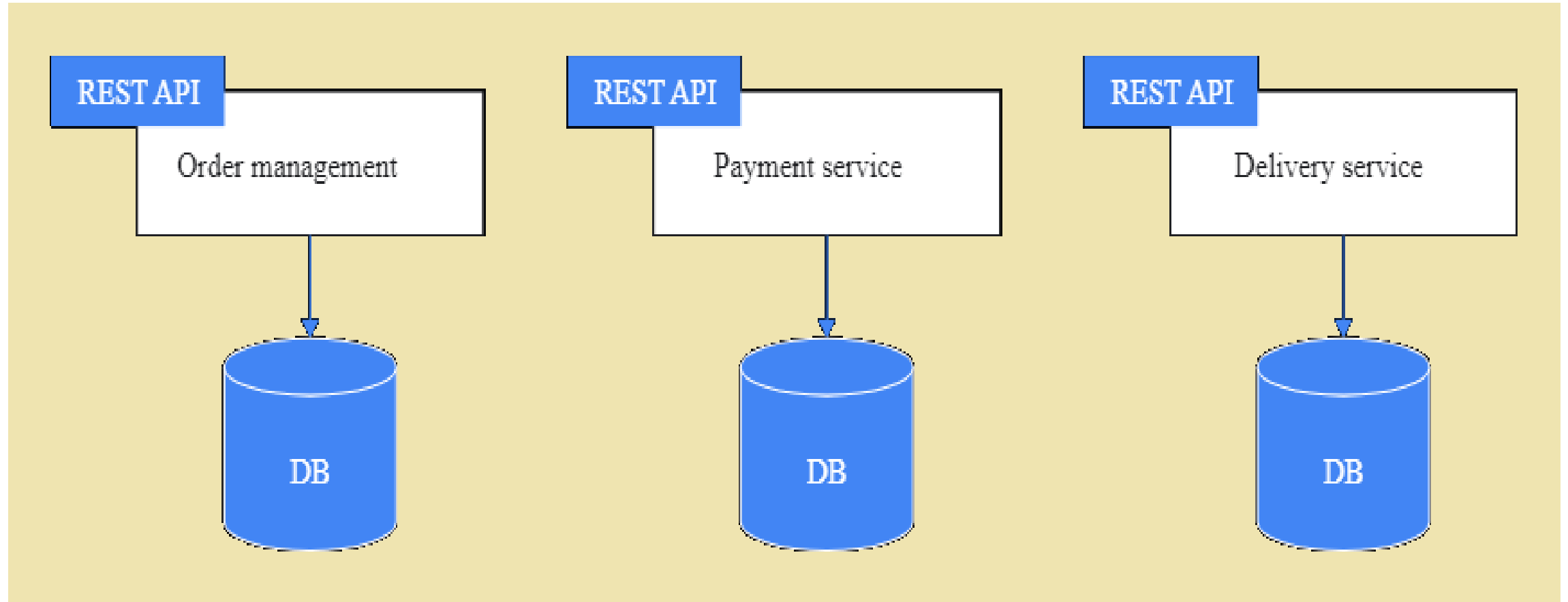
- Bounded by contexts

- It encapsulates the details of a single domain, and define the integration with other domains.
- It is about implementing a business capability

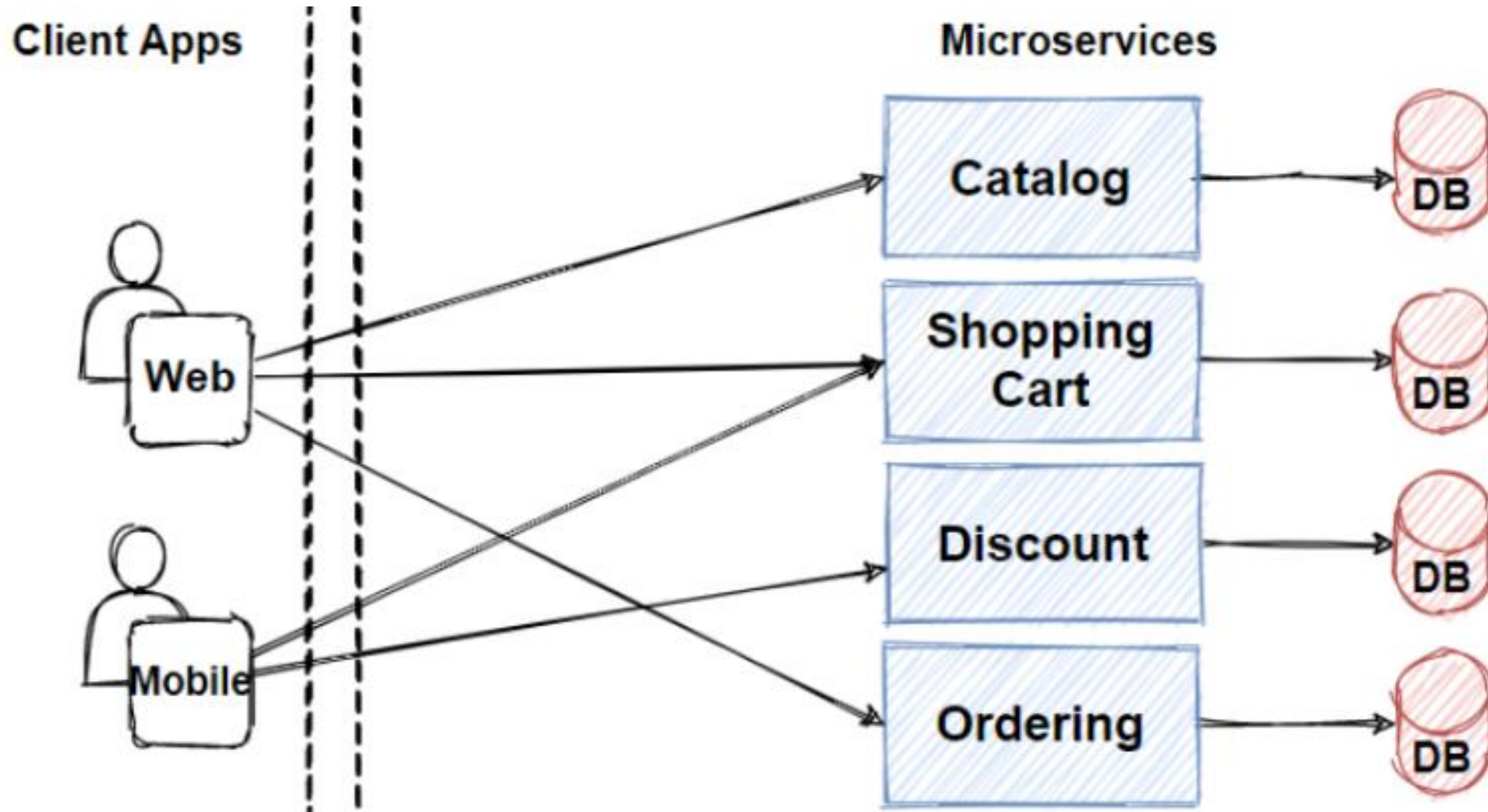
# E-Commerce Application with MSA



# Each Service in MSA has its own database



# Each Service in MSA has its own database





# Benefits of Own Database for Each Service

- **Loose coupling**
  - If two services share the same database:
    - A schema change in one service can break another
    - Services become **tightly coupled** again (like a hidden monolith)
  - Owning its own database keeps a service **independent**.
- **Independent deployment & scaling**
  - Each microservice can:
    - Choose its own database type (SQL / NoSQL)
    - Scale database based on its own load
  - One service's heavy queries don't affect others

# Benefits of Own Database for Each Service

- **Clear data ownership**
  - Each microservice is the **single source of truth** for its data.
  - Example:
    - Order Service owns orders
    - User Service owns user data
  - Other services **cannot directly access** that data.
- **Fault isolation**
  - If one database goes down:
    - Only that service is affected
    - The whole system doesn't collapse

# Micro services NEVER share databases directly

- **API calls (Request–Response)**
  - A service requests data from the owning service.
  - **Example:**
    - Order Service needs user info
    - It calls → User Service API
  - Most common approach
  - Adds runtime dependency

# Micro services NEVER share databases directly

- **Event-driven communication (recommended)**
  - Services publish events when data changes.
  - **Example:**
    - User Service publishes UserCreated event
    - Order Service stores required user data locally
  - Loose coupling
  - Better scalability
  - Eventual consistency

# Micro services NEVER share databases directly

- **Data replication (Read models)**
  - A service stores **only the data it needs**, not full data.
  - **Example:**
  - Order Service stores:
    - UserId
    - UserName (copied)
  - This is called **denormalization**.

# Micro services NEVER share databases directly

- **CQRS (Command Query Responsibility Segregation)**
  - One service **writes** data
  - Multiple services **read** optimized copies
- Used in high-scale systems.
- **Important concept: Eventual Consistency**
  - Because databases are separate:
    - Data is **not always immediately consistent**
    - System becomes consistent **over time**
- This is **acceptable and expected** in microservices.

# Benefits of Micro services Architecture

- **Asynchronicity** – Micro services communicate asynchronously using messages or events, allowing services to operate without waiting for immediate responses.
- **Independent Deployment** – Each microservice can be deployed, updated, or rolled back independently without affecting other services.
- **Evolutionary Architecture Style** – The system can evolve gradually by modifying or replacing individual services instead of redesigning the entire application.
- **Components are Deployed** – Application functionality is split into small components (services) that are deployed as separate units.

# Benefits of Micro services Architecture (..cont)

- **Features are Released** – New features can be released service-by-service rather than as a single large application release.
- **Applications Consist of Routing** – Requests are routed to appropriate micro services through APIs or gateways based on functionality.
- **Scalability** – Individual services can be scaled independently based on demand, improving resource utilization.
- **Horizontal Scaling (Adding More Instances)** – Scalability is achieved by running multiple instances of the same microservice instead of increasing server size.
- **Workload Partitioning** – Application workload is distributed across multiple services or instances to improve performance and reliability.



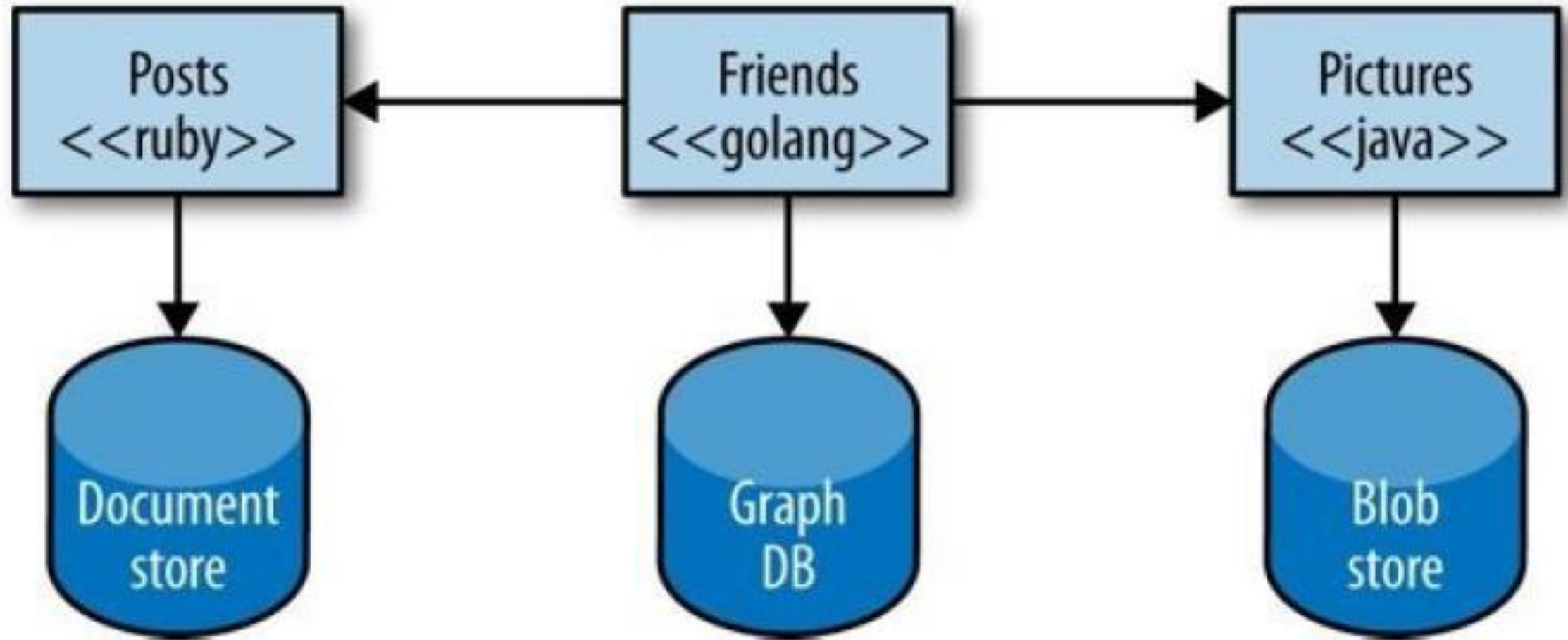
# Benefits of Microservices Architecture (..cont)

- **Easier to understand the code** – It is easy to distinguish one small service and flow of the whole service rather than one big code base
- **Fast Software delivery** – Each service can be developed by different developers and in many different languages
- **Efficient debugging** – Don't have to jump through multiple layers of an application and in essence better fault isolation
- **Reusable** – Since it is an independent service it can be used in other projects also

# Benefits of Microservices Architecture (..cont)

- Don't have to **scale** the whole project, only need to scale up that component which needs to scale up
- **Re-Deployment** – Need only to deploy that service which has been changed not the whole project again

# Technology Heterogeneity (Social Network App)



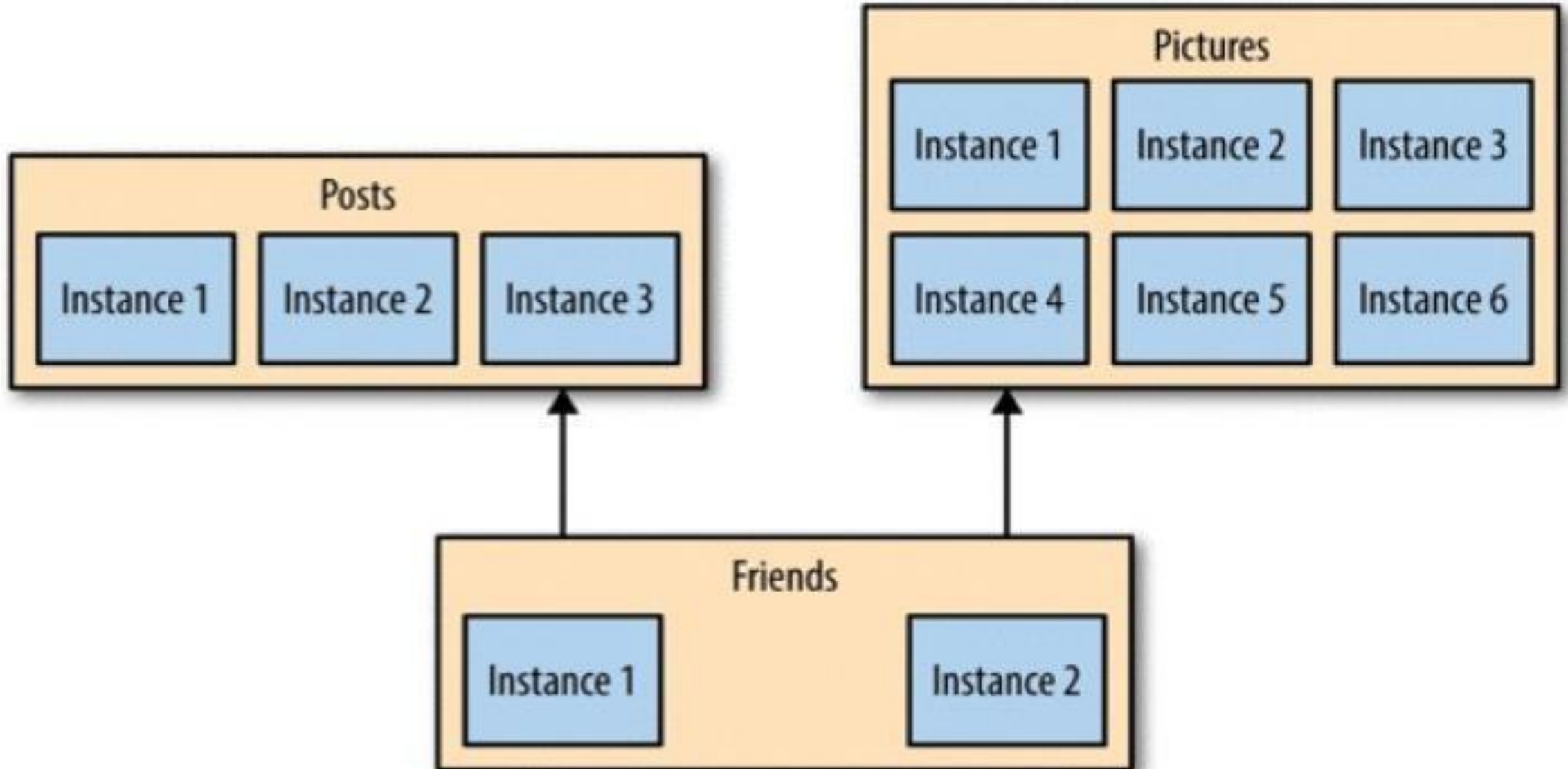
# Resilience

- If one component of a system fails, but that **failure doesn't cascade**, you can isolate the problem and the rest of the system can carry on working
- In a **monolithic service**, if the service fails, everything stops working
- With a **monolithic system**, we can run on multiple machines to reduce our chance of failure
- With microservices, we can build systems that **handle the total failure** of services and degrade functionality accordingly
- With Microservices we need to understand the new sources of failure that distributed systems have to deal: **network/machines failure**

# Scaling

- With a large, monolithic service, we have to **scale everything together**
- One small part of our overall system is constrained in performance, but if that behavior is locked up in a giant monolithic application, we have to handle scaling everything as a piece
- **With smaller services**, we can just scale those services that need scaling
- It allows us to run other parts of the system on smaller, less powerful hardware

# Scaling



# Ease of Deployment

- A **one-line change** to a million-line-long **monolithic application** requires the whole application to be re-deployed in order to release the change
- That could be a large-impact, high-risk deployment
- With **microservices**, we can make a change to a **single service** and deploy it independently of the rest of the system
- This allows us to get our code deployed faster
- If a problem does occur, it can be **isolated quickly** to an individual service, making fast rollback easy to achieve
- It also means we can get our new functionality out to customers faster

# Organizational Alignment

- There are problems associated with **large teams** and large codebases
- These problems can be aggravated when the team is distributed
- **Smaller teams** working on smaller codebases tend to be more productive
- Micro services allow us to better align our architecture to our organization
- It helps us to minimize number of people working on any codebase as per the team size and productivity

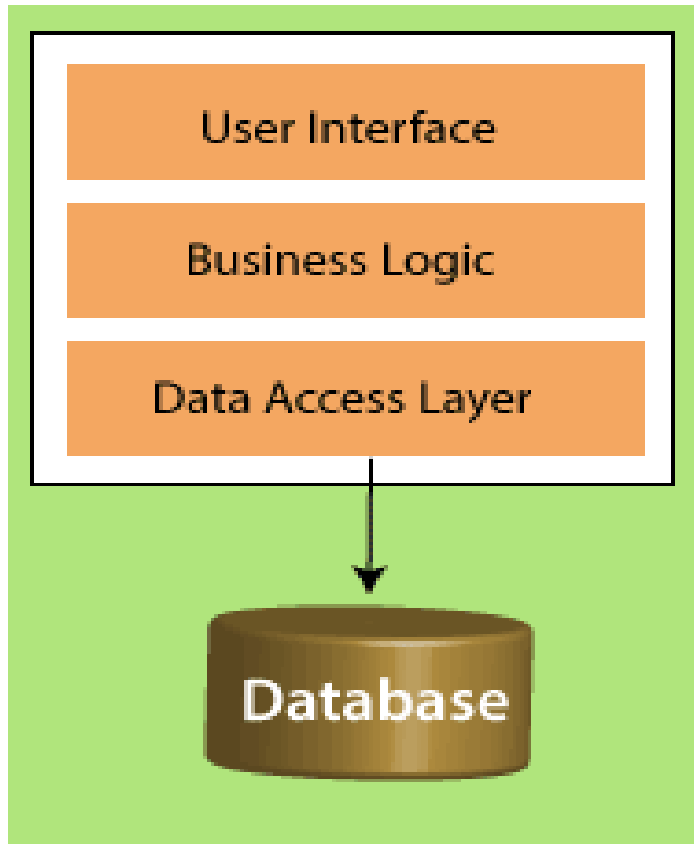


# Composability

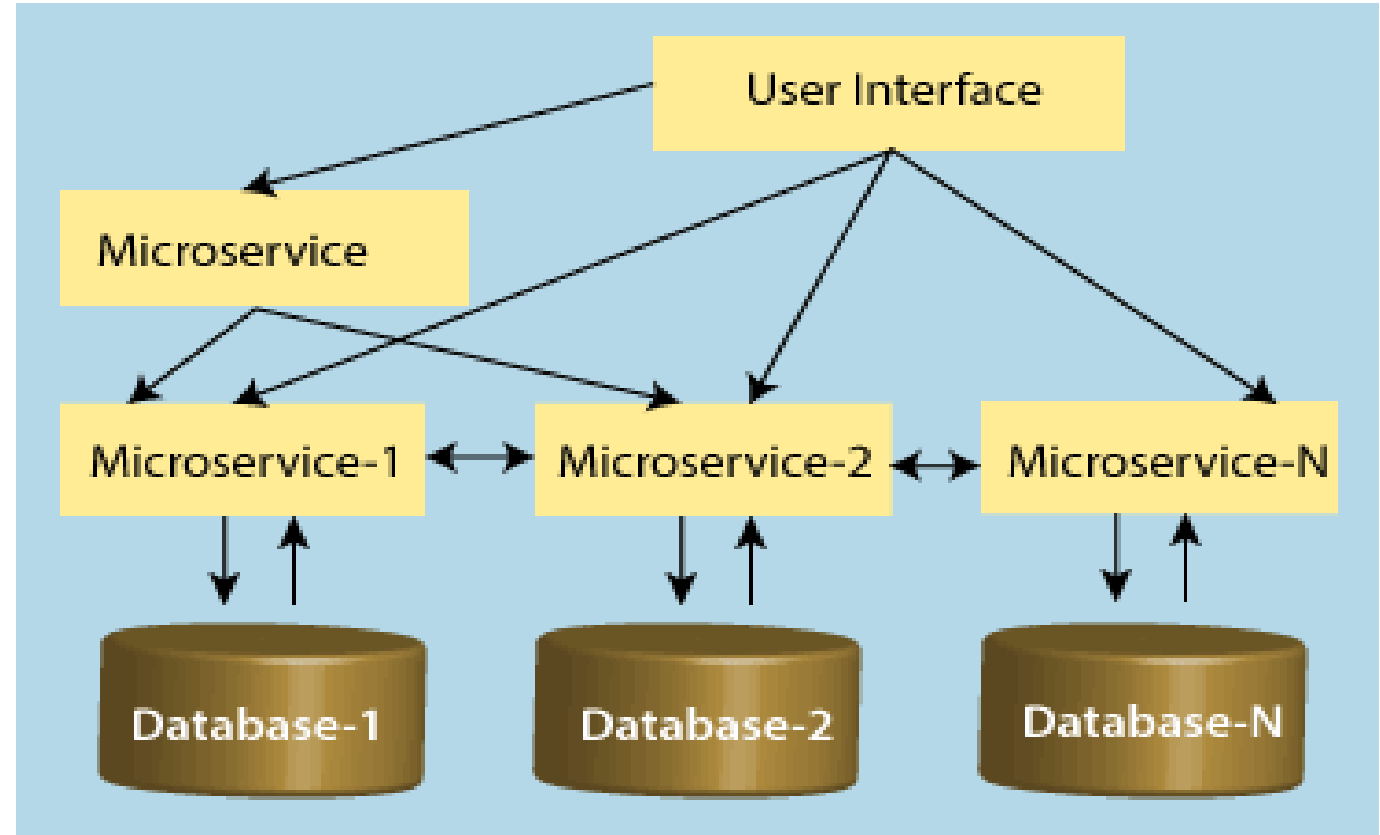
- With a **monolithic application**, we have one **coarse-grained seam** (one big access point) that can be used from the outside
- With **microservices**, we allow for our functionality to be consumed in **different ways** for different purposes
- Web, native application, mobile web, tablet app, or wearable device

# Optimizing for Replaceability

- Microservices being small in size, the cost to replace them with a better implementation, or even delete them altogether, is much easier to manage
- The barriers to rewriting or removing services entirely are very low



Monolithic Architecture



Microservice Architecture

Monolithic vs Microservice Architecture

# Challenges of Microservices Architecture

- Microservices have all the associated **complexities** of the distributed system
- There is a higher **chance of failure** during communication between different services
- Difficult to **manage** a large number of services
- The developer needs to solve the problems such as **network latency** and **load balancing**
- **Complex testing** over a distributed environment

# SOA vs. Micro services

- **Size Difference:**

- The size difference is based on the granularity of the components inside the architectures.
- The components or services in Micro service architecture are comparably much smaller and lighter than that of Service-oriented architecture.

- **Scope Difference:**

- SOA includes loads of business features and functionality and is used as a complete subsystem most of the time, while MSA perform the single task that they are designed to perform.

# SOA vs. Micro services

- **Data Storage Difference:**

- The services in Service Oriented Architecture share common data storage most of the time.
- In the case of MSA, there can be an independent data storage section for each service.

- **Development Dependency:**

- Both Service Oriented Architecture and Micro services are built to be managed by several teams, but in the case of Service Oriented Architecture, the communication mechanism needs to be common for all teams.
- While in Micro services, each team can work on distinct service that can be developed and deployed without the interference of other services

# SOA vs. Micro services

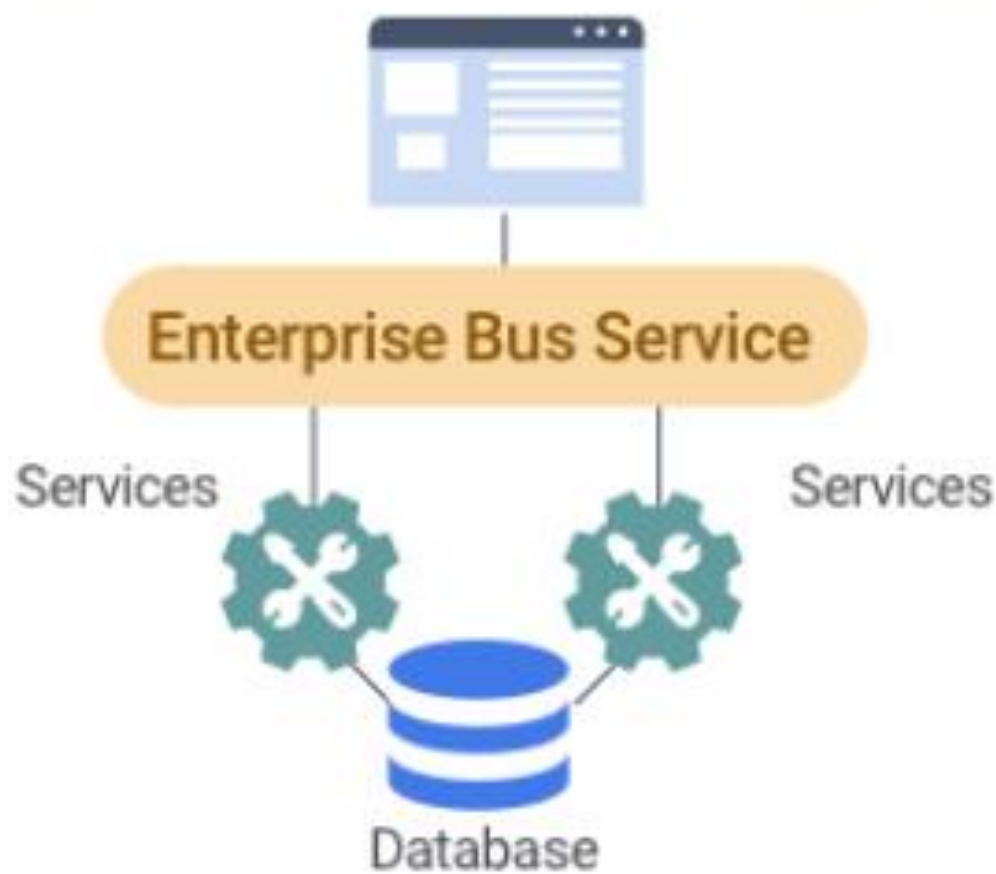
- **Remote Accessing Protocols:**
  - SOA use Simple Object Access Protocol (SOAP) or messaging mostly for remote access, while Microservice architecture relies mostly on REST and sometimes does the work through simple messaging such as MSMQ.
- **API layer vs. Messaging middleware:**
  - Service Oriented Architecture has a component named messaging middleware, while Microservice architecture uses the API layer.
  - The messaging middleware has some benefits over the API layer, such as enhancement of message, protocol transformation, and routing.
  - The API layer is present between service consumers and service in the case of Microservice architecture.

# SOA vs Microservices



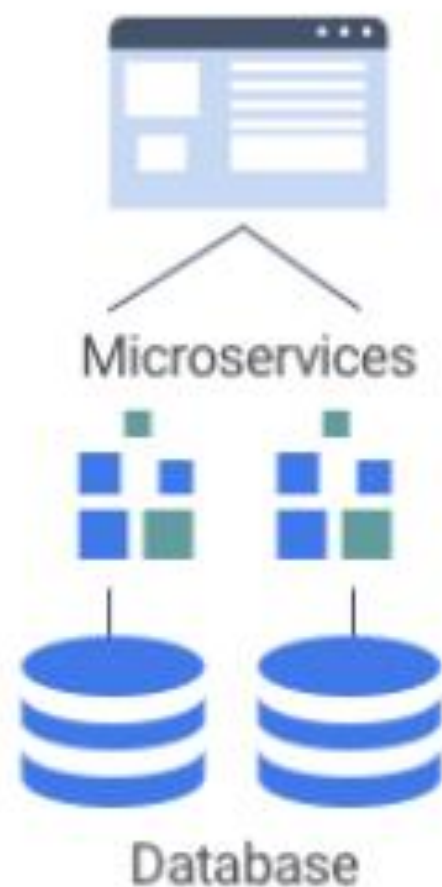
XENONSTACK

## Service Oriented Architecture



**Vs**

## Microservices





# Enterprise Service Bus (ESB)

- It is a **middleware tool** used to **distribute work** among connected components of an application
- Designed to provide a uniform means of moving work, offering applications the ability to **connect to the bus** and **subscribe to messages** based on simple structural and business policy rules
- It's a tool that has use in both distributed computing and component integration
- Visualize it as a **set of switches** that can direct a message along a specific route between application components based on message contents and implementation or business policies

# Enterprise Service Bus (ESB)

- It is the **centre** of application workflow
- It is, in effect, a **message queue** that handle information exchanges throughout the application
- It does not dictate whether components that use the bus are local to it or remote, nor does it enforce any specific requirements for programming languages
- It acts to unify the various ways in which components can receive or send information to other application elements

# Enterprise Service Bus (ESB)

- It makes **easy to change** components or **add** additional components to an application
- It also makes for a convenient place to enforce **security and compliance** requirements, log normal or exception conditions and even handle transaction performance monitoring
- It also provides **load balancing** in which multiple copies of a component can be instantiated to improve performance
- It can also often provide **failover support** should a component or its resources fail

# API Gateway

- An API gateway is an **API management tool** that sits between a client and a collection of backend services
- An API gateway acts as a **reverse proxy** to accept all application programming interface (API) calls, aggregate the various services required to fulfil them, and return the appropriate result

 **Client**

**API  
Gateway**

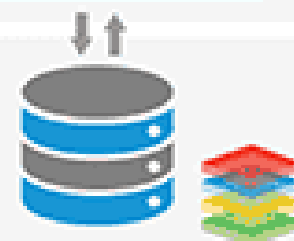
  
**Microservice**

  
**Microservice**

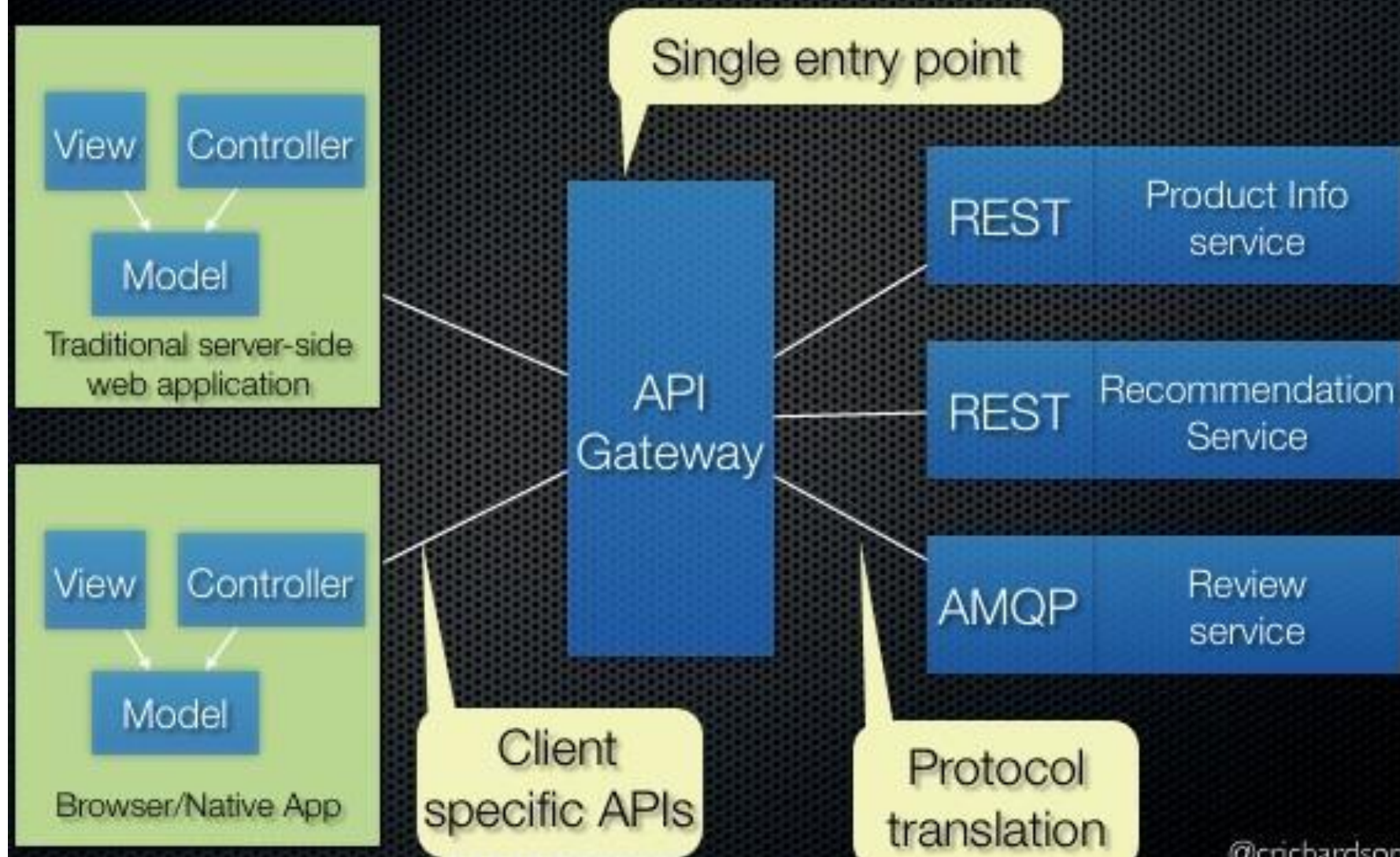
  
**Microservice**

  
**Microservice**

  
**Microservice**



# Use an API gateway



# Why use an API Gateway?

- **Abstracting** the clients from the complexity of the backend system
- Handling **Authentication and Authorization** of users in a single place for the entire system
- **Transforming** the requests and responses according to the format needed
- **Rate limiting** for each of the APIs to prevent the load on backend services
- **Throttling** the API requests made by a single user to prevent the DDOS attack
- Reducing the load on the services by **Caching** the responses for the repeated requests

# Why use an API Gateway?

- Supports **different protocols**
- Metering individual APIs to make **billing** the clients based on usage
- **Logging** and tracing the API calls into the system in a single place
- Error handling
- Aggregating the data across microservices and respond the data to the client on a single API call. This will avoid multiple API calls the clients to need to make



# Drawbacks of an API Gateway

- It can become a **single point of failure**
- It could cause performance issues and **increase the latency** of API calls if we perform too many things in the API Gateway
- If we have too many microservices, **managing** too many routes could get complicated
- There is always an **extra hop** on API calls from the client to the server
- **Extra cost** because of the additional server component involved

# Backend for Frontend (BFF) Pattern

- The BFF pattern is an architectural paradigm, a variant of the API gateway pattern
- It comprises of multiple back-ends that are designed to meet the demands of specific front-end applications, such as desktop, browser, and native-mobile applications, **IOT** devices etc.

# Backend for Frontend (BFF)

- Instead of **one generic backend for all clients**, we create:
  - One backend for **Web**
  - One backend for **Mobile**
  - One backend for **Tablet / IoT**, if needed
- Each backend serves **only one frontend** and returns **exactly the data format it needs**.

# Example

- **Example (E-commerce)**
- **Web BFF:**
  - Returns full product details, reviews, recommendations
- **Mobile BFF:**
  - Returns product name, price, thumbnail only

# Why Backend for Frontend (BFF)?

- Different frontends have **different requirements**:
  - Mobile needs **less data** (bandwidth-sensitive)
  - Web can handle **richer responses**
  - **UI flows** differ (screens, navigation)
- A **single backend API** becomes:
  - Overloaded with conditional logic
  - Hard to maintain
  - Inefficient for some clients

# Steps for Implementation

- **Identify Different Frontends:**

- Analyze the **needs** of each client (e.g., web, mobile, IoT) and determine how their data and interaction requirements differ.
- This will help to decide if separate backends are needed for each frontend.

- **Design Custom APIs for Each Client:**

- Create **custom API** endpoints for each frontend.
- Tailor these APIs to provide only the necessary data and logic required by that specific client.
- For example, a mobile API might return lighter payloads compared to a web API.

# Steps for Implementation

- **Implement Backend Services:**

- Build a **dedicated backend** for each client (web, mobile, etc.).
- Each BFF should aggregate, process, or transform data from core services, making the responses optimal for its respective client.

- **Integrate Core Services:**

- The BFF should communicate with **existing** microservices or monolithic backends to fetch, transform, and combine data.
- Ensure that your core services remain client-agnostic, while BFF handles frontend-specific needs.

# Steps for Implementation

- **Handle Frontend-Specific Business Logic:**
  - Move any complex business logic or data transformation (that would otherwise reside in the frontend) into the BFF.
  - This **simplifies frontend applications** by reducing data processing and logic.
- **Optimize Performance:**
  - Implement **caching, pagination, and response optimizations** in the BFF to reduce latency and improve client performance.
  - For mobile or IoT, minimize payload size and avoid sending unnecessary data.

•



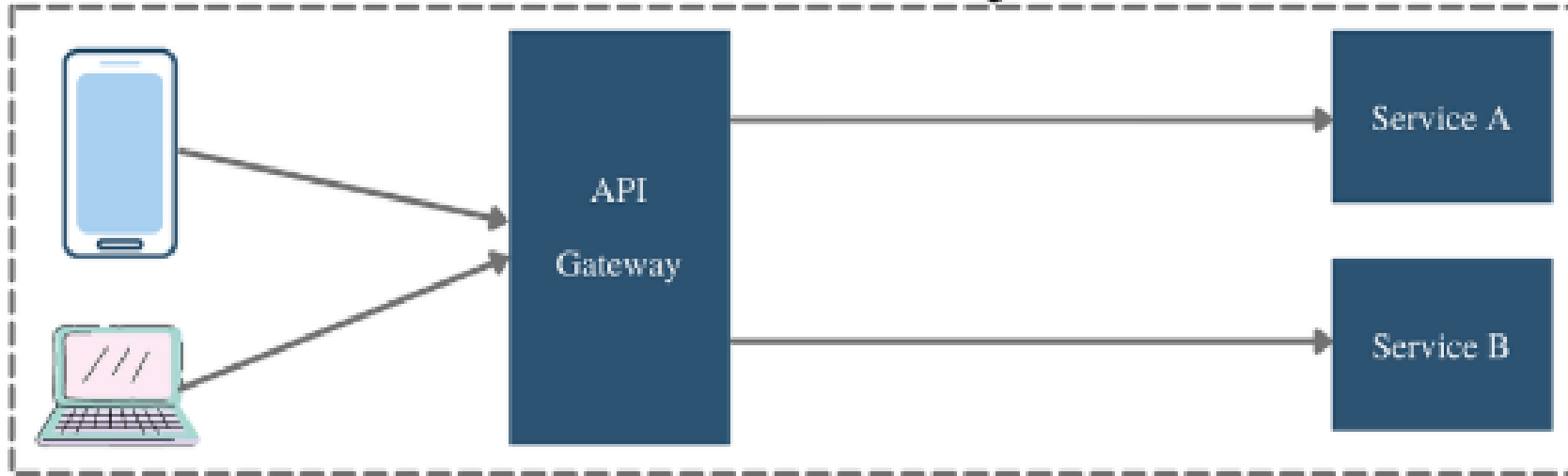
# Steps for Implementation

- **Enforce Security:**
  - Set up **client-specific** security policies (e.g., authentication, rate limiting, data filtering) in each BFF.
  - Ensure the backend for each client has tailored security to meet its requirements.
- **Monitor and Maintain BFF Services:**
  - Monitor the **performance and health** of each BFF to ensure it functions effectively.
  - **Maintenance and scaling** should be managed individually for each BFF, allowing frontends to evolve independently.

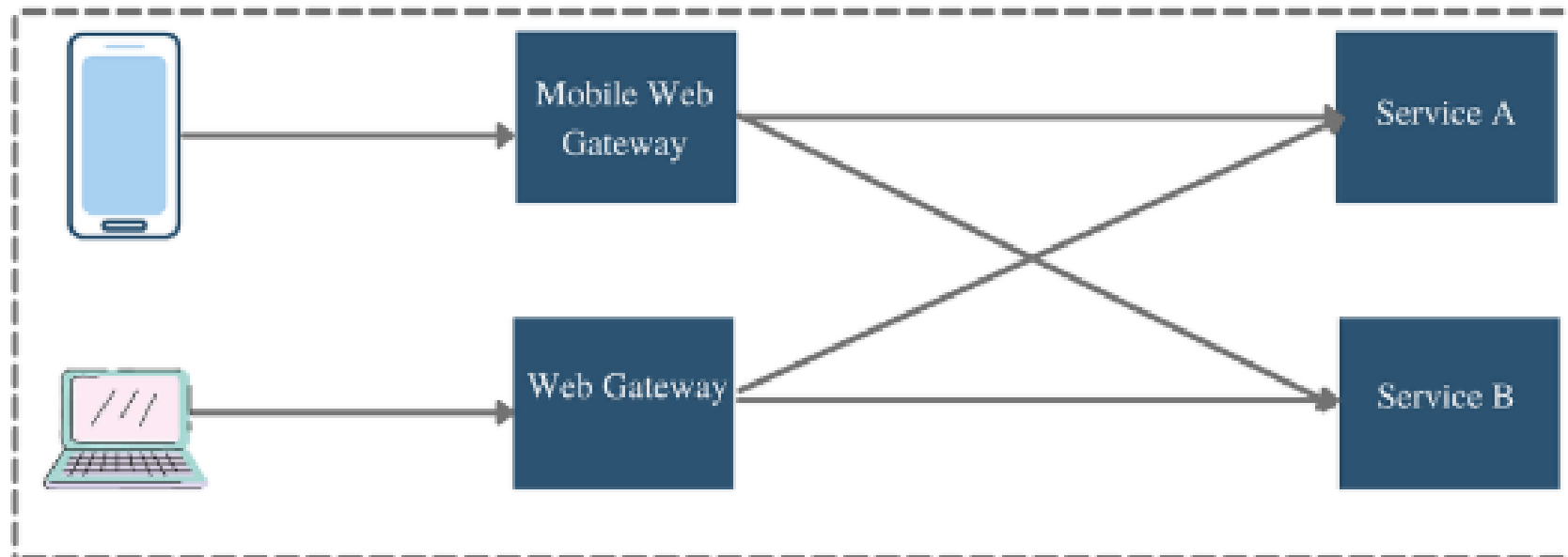
# Steps for Implementation

- **Test and Deploy:**
  - Test each BFF **separately** to ensure it works well with its corresponding frontend.
  - Once ready, deploy the BFF services, keeping them isolated so that changes to one frontend don't impact others.
- **Iterate and Evolve:**
  - As the frontends evolve, update the BFF accordingly, adding new features or optimizations specific to each client.

## API Gateway



## Backend For Frontend



# Benefits

- Optimized APIs per frontend
- Reduced over-fetching / under-fetching
- Cleaner frontend code
- Better performance
- Faster frontend development
- Better Scalability
- Enhanced Security

# Challenges

- Increased Complexity
- Higher Maintenance Costs
- Duplication of Logic
- Versioning and Dependency Management
- Scalability and Performance
- Latency and Overhead
- Communication Complexity

# Real-World Use Cases for BFF Pattern

- Spotify
- Netflix
- Airbnb
- Amazon

| Microservice Based Architecture                                                                                                                    | Service-Oriented Architecture                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Uses <b>protocols</b> such as <b>REST</b> , and <b>HTTP</b> , etc.                                                                                 | SOA supports <b>multi-message protocols</b> .                                        |
| It focuses on <b>decoupling</b> .                                                                                                                  | It focuses on application service <b>reusability</b> .                               |
| It uses a <b>simple messaging system</b> for communication.                                                                                        | It uses <b>Enterprise Service Bus (ESB)</b> for communication.                       |
| follows " <b>share as little as possible</b> " architecture approach.                                                                              | follows " <b>share as much as possible architecture</b> " approach.                  |
| much better in <b>fault tolerance</b>                                                                                                              | not better in fault tolerance in comparison to MSA.                                  |
| Each microservice have an <b>independent</b> database.                                                                                             | SOA services share the <b>whole</b> data storage.                                    |
| MSA used <b>modern</b> relational databases.                                                                                                       | SOA used <b>traditional</b> relational databases.                                    |
| MSA tries to <b>minimize</b> sharing through bounded context (the coupling of components and its data as a single unit with minimal dependencies). | SOA <b>enhances</b> component sharing.                                               |
| It is better suited for the <b>smaller</b> and <b>well portioned</b> , web-based system.                                                           | It is better for a <b>large</b> and <b>complex</b> business application environment. |